INTERNAL REVIEW

TimeFrame (Lincoln): Code Review and Speed Plan

Prepared by Mohsin Naqvi, Kwanso | November 4, 2026 | Confidential, internal to the Kwanso project team

Scope. The code in `timeframe-prod_Lincoln2025-master`: a React/TypeScript client and a Python FastAPI server. This is the repository the team is working in.
What I did. Read the whole conversation pipeline, ran the app locally in Docker, and measured stage timings from the project's own log database (`server/logs.db`, recorded by the original developer).
What I could not do. Run a live conversation, because the API keys are not set up yet. The timings below therefore come from those logs, not from my own runs.

1. Summary

- **The client's complaint is confirmed.** In the logged conversations, Lincoln starts speaking about **7.5 seconds after the visitor stops talking**, plus however long speech recognition takes to decide the visitor has finished (that step is not logged).
- **Where the time goes:** the full LLM answer is generated before any speech starts (median 2.95 s), the D-ID avatar video takes time to start (median 3.09 s), and the code waits a fixed 1.5 s on every turn. The tuned "visitor has finished speaking" settings are also never applied.
- **Quick, low-risk changes** save about 1.5 s on every turn, and about 6 s on turns where the visitor's words were empty or unclear (5 of the 11 logged turns). The largest further gain needs a short prototype: stream the answer to the avatar one sentence at a time.
- **Other issues found:** stability and privacy problems (the conversation history carries over to the next visitor, the server freezes during every LLM call, the WebSocket mishandles disconnects, Stop can hang) and gaps against the client brief (no local LLM, and offline mode is a placeholder).

2. How a conversation works today

#	Step	Runs in	What happens
1	Start	Browser	Staff click the hidden top-left area, which calls `/start-loop`.
2	Presence detection	Server	The browser sends 8 webcam frames per second over a WebSocket. YOLO finds a face, then MediaPipe checks that the person is looking at the screen. After 1 s of looking, the server fetches an AssemblyAI token and sends `start-listening`.
3	Listening	Browser → AssemblyAI (cloud)	The browser streams the microphone to AssemblyAI. When AssemblyAI marks the end of the visitor's turn, the browser waits 1.5 s and then sends the text to the server.
4	Thinking	Server → OpenAI (cloud)	`gpt-4o` generates the whole answer using the Lincoln system prompt (about 1,600 words). DeepRails then checks the answer in the background.
5	Speaking	Browser → D-ID (cloud)	The full answer is sent to the D-ID avatar, which speaks it. Subtitles are animated at a fixed 165 words per minute.

6	Loop	Browser	Once both the video and the subtitles finish, the browser fetches a new token and listens again. If nobody speaks for 8 s, it returns to presence detection.

3. Where the time goes (measured)

Stage	Time	Source
AssemblyAI decides the visitor has finished speaking	Not logged	The browser uses AssemblyAI's default settings (see 4.2)
Fixed wait before sending the text to the server	**1.5 s** on every turn	`client/src/hooks/_useAAI.tsx:84`
LLM answer (`gpt-4o` , not streamed)	0.97–4.73 s, **median 2.95 s** (11 turns)	logs.db: `start-thinking` → `start-speaking`
D-ID avatar video starts	2.16–4.00 s, **median 3.09 s** (6 turns)	logs.db: `start-speaking` → `speaking`
Total wait the visitor sees	**≈ 7.5 s** + end-of-turn time	Sum of the medians above

Caveat: this is a small sample, mostly from 17 October 2025, and the code may have changed since. Treat it as indicative. The first task in the plan (section 7) is to re-measure on the current code.

Wasted turns: in 5 of the 11 logged conversations the transcribed question was empty or meaningless (for example "windows ten commas library"). Each still went through a full LLM and avatar cycle, about 6 s, only for Lincoln to answer "Pardon me, I could not quite make sense of that…".

4. Speed plan, ranked

Size: S = up to about a day, M = a few days, L = more. Gains are estimates unless marked "certain". Every change will be checked with the measurement plan in section 7.

Phase 1: quick wins (low risk)

#	Change	Where	Expected gain	Size	Risk / note
4.1	Remove the fixed 1.5 s wait before sending the visitor's text to the server.	`_useAAI.tsx:84`	**1.5 s every turn** (certain)	S	Low. Check that it isn't hiding a timing race.
4.2	Apply the end-of-turn settings to the browser's AssemblyAI connection (`endOfTurnConfidenceThreshold` , `minEndOfTurnSilenceWhenConfident` , `maxTurnSilence` , all present in the installed SDK v4.17.0). Today these values exist only in `server/config/stt.py` , which is used only by the disabled server-side listener.	`_useAAI.tsx:46`	Unknown until measured. Also the main lever for the client's "answers before I finish" complaint.	S	Medium. Shorter timeouts cut visitors off, longer ones add delay. Needs tuning in a realistic room.

4.3	Don't call the LLM or the avatar for empty or very short transcripts. Keep listening, or play a short pre-recorded "please repeat" clip.	`_useAAI.tsx`, `thinking/index.py`	About **6 s** on those turns (5 of 11 logged)	S	Low.
4.4	Start listening again as soon as the avatar video ends, without waiting for the subtitle animation, and fetch the next AssemblyAI token while Lincoln is still speaking.	`Main.tsx:30–56`	Faster return to listening (not measured)	S	Low.
4.5	Remove leftover debug logging: a `console.log` on every audio chunk and every render, and a `print` on every detection loop.	`_useAAI.tsx:171`, `Text.tsx`, `presence_detection/index.py`	Less CPU use	S	None.

Phase 2: bigger gains (needs a prototype)

#	Change	Where	Expected gain	Size	Risk / note
4.6	Stream the LLM answer and send the first complete sentence to D-ID straight away, queuing the rest.	`llm_online.py`, `useVideoProviderService.tsx`	Lincoln starts after the first sentence instead of the whole answer. Likely the largest single gain.	M	Medium. Need to confirm that D-ID plays back-to-back `speak()` calls smoothly.
4.7	Benchmark faster models, including the local LLM the brief requires, on the same prompt and questions. Check whether prompt caching applies to the ~1,600-word system prompt sent with every request.	`config/thinking.py`	To be measured	M	Answer quality and persona fidelity must be checked, not just speed.
4.8	Test the D-ID output resolution at 720 instead of 1080.	`useVideoProviderService.tsx:113`	Unknown, test only	S	Visual quality trade-off.
4.9	Run the LLM call off the server's event loop (`asyncio.to_thread`).	`websocket_manager.py:67`	Stability (see 5.2)	S	Low.

Phase 3: platform work

#	Change	Why	Size
4.10	Rework presence detection: process only new frames at the camera's rate, and replace YOLO with MediaPipe face detection (MediaPipe is already used for the "looking at the screen" check).	Lower CPU use, faster start, removes `torch` / `ultralytics` (a very large install, and AGPL-3.0 licensed).	M

4.11	Evaluate local speech-to-text and avatar options, if the client confirms the local/offline requirement in the brief.	Removes cloud round-trips and per-use costs.	L

5. Other problems found

#	Severity	Problem	Where	Impact and fix
5.1	**High**	The conversation history is never reset between visitors.	`thinking/index.py` (global `chat`)	The previous visitor's questions and answers are sent to the LLM with the next visitor's question. This affects answers and is a privacy issue in a public venue. **Fix:** clear the history when a new session starts.
5.2	**High**	The LLM call blocks the server's event loop.	`websocket_manager.py:67`	While `gpt-4o` answers (~3 s), the server cannot process camera frames or any other message. **Fix:** run the call in a worker thread.
5.3	**High**	WebSocket disconnects are not handled.	`websocket_manager.py:46`	Every page reload or close raises a `RuntimeError` (seen in our Docker run). The dead connection stays in the list and the `connected` flag goes stale, so `/start-loop` can wrongly accept or reject. **Fix:** handle the `websocket.disconnect` message.
5.4	**High**	Stop can hang.	`detect_person.py:6`	If no camera frame arrives (for example, camera permission denied), this loop spins forever and ignores the stop signal, and `/stop-loop` waits on it. **Fix:** wait with a timeout and check the stop signal.
5.5	**Medium**	Face detection runs in a busy loop.	`presence_detection/index.py`	Runs YOLO again and again on the same frame with no pause, so CPU is near 100% even while idle. **Fix:** wait for a new frame.
5.6	**Medium**	The visitor's question is sent to the LLM twice.	`thinking/index.py:17` + `llm_online.py:18`	It is added to the history and then appended again. Extra tokens, slightly slower. **Fix:** add it once.
5.7	**Medium**	The 1 s "looking at the screen" timer is not reset after a session.	`presence_detection/index.py:27–35`	The next detection can trigger listening immediately, without the 1 s look. **Fix:** reset the timer when the loop starts.

5.8	**Medium**	Offline mode is a placeholder.	`utils/internet.py:16`, `ll_offline.py:24`	The internet check always returns True, and the offline "LLM" always returns "1". There is no working local or offline path.
5.9	**Medium**	D-ID errors in the logs.	logs.db	214 "Failed to fetch" errors (Oct 2025; source not recorded) and 18 "Agent not found" errors (Apr 2026). Confirm which D-ID account and agent are current.
5.10	**Medium**	The WebSocket URL is hard-coded.	`client/src/apis/socket.ts:4`	`ws://127.0.0.1:8000/ws` ignores `VITE_API_BASE_URL`, which blocks any non-local deployment.
5.11	**Low**	The last camera frame is never cleared, and the camera stream is never released.	`frame_buffer.py`, `useCameraSender.tsx:67`	An old frame can count as "face present". The cleanup function is returned inside an async block, so it is never registered.
5.12	**Low**	A temporary AssemblyAI token is written to the logs.	`Main.tsx:83`	The token is short-lived, but it shouldn't be logged.
5.13	**Low**	Dead code.	Server: `stt_service.py`, `mic_manager.py`, `camera_manager.py`, `stt_offline.py`. Client: `useAAI.tsx`, `__useVideoProviderService.tsx`, `WebcamView.tsx`	Confusing for maintenance. Remove or archive after confirming with the client.
5.14	**Low**	Setup and build problems.	README, Dockerfiles, `requirements.txt`	The README names the wrong client env key. The Dockerfiles were broken: a missing `deeprails` package, Node 18, a dead model download link, Linux-only device settings, and TypeScript build errors (all fixed locally). `requirements.txt` is saved in UTF-16. `mediapipe 0.10.21` has no Linux ARM build. The MySQL service is unused (logs go to SQLite).

6. Gaps against the client brief

- **Local LLM:** the brief describes one Mac Mini per display running a local LLM. This code uses OpenAI `gpt-4o` in the cloud, and the local path is a placeholder (5.8).
- **Cloud dependencies:** OpenAI, AssemblyAI, D-ID and DeepRails, each needing an internet connection, an API key and a paid account. We don't know whose accounts they are; this is an open question for Erick.
- **Native macOS front end:** the brief expects Swift. This code has no Swift; the app runs in a browser.

- **Persona platform:** Lincoln's persona is hard-coded in `server/config/thinking.py`. There is no persona configuration model yet.
- **Licensing:** `ultralytics` (YOLOv8) is AGPL-3.0, which must be disclosed under the brief and reviewed before commercial use.
- **Linux:** not tested on Ubuntu or Kubuntu.

7. How we will measure

1. Add one timestamped log entry per stage to the existing logs table: end of turn received, text sent to server, LLM first token, LLM done, avatar `speak()` called, video starts, video ends, listening reopened.
2. Run a fixed set of 20 test questions on the current code to get a baseline, then repeat after each phase.
3. Agree target times with the client. The brief leaves the "conversation timing" thresholds open for the assessment phase.

8. Limits of this review

- Timings come from the original developer's logs (small sample), not from live runs on the current code.
- My local Docker run used amd64 emulation on Apple Silicon, so its speed is not representative of a Mac Mini.
- No API keys were available, so the listening, LLM and avatar steps were reviewed from the code only.

9. Proposed next steps

1. Get API keys or test accounts (OpenAI, AssemblyAI, D-ID).
2. Add the stage timing logs and record a baseline (section 7).
3. Ship Phase 1 together with the high-severity fixes (5.1–5.4), then re-measure.
4. Prototype sentence-by-sentence streaming to D-ID (4.6) and benchmark the model options (4.7).
5. Report the before-and-after numbers and agree on Phase 3 scope with the client.

Confidential. Internal to the Kwanso project team. All file references are relative to the project root. Gains marked as estimates have not been measured.